

Book Metadata, Review & Insight API - API Documentation

1. Overview

- **Local base URL:** <http://127.0.0.1:8000>
- **Deployed base URL:** <https://book-insight-api.onrender.com>
- **API prefix:** /api/v1
- **Authentication:** Bearer token in the Authorization header
- **Interactive documentation:**
 - Local Swagger UI: <http://127.0.0.1:8000/api/v1/docs>
 - Live Swagger UI: <https://book-insight-api.onrender.com/api/v1/docs>
 - Live ReDoc: <https://book-insight-api.onrender.com/api/v1/redoc>
- **Bundled public dataset:** `data/raw/open_library_books.csv`, generated from the Open Library Search API

1.1 Data source note

The project includes a real public metadata export rather than a hand-written placeholder CSV. The bundled file `data/raw/open_library_books.csv` is a curated subset of Open Library data mapped into the schema expected by `scripts/import_books_from_csv.py`.

1.2 Error response format

All business and validation errors are normalised to a consistent top-level shape:

```
{
  "error": {
    "code": "BOOK_NOT_FOUND",
    "message": "Book not found",
    "details": {
      "book_id": 1
    }
  }
}
```

Common error codes include `USER_EXISTS`, `INVALID_CREDENTIALS`, `AUTHOR_NOT_FOUND`, `BOOK_NOT_FOUND`, `BOOK_ISBN_EXISTS`, `REVIEW_NOT_FOUND`, `REVIEW_FORBIDDEN`, `READING_LIST_ENTRY_NOT_FOUND`, `READING_LIST_FORBIDDEN`, and `INTERNAL_SERVER_ERROR`.

2. Health

Method	Path	Description
GET	/api/v1/health/ps	Simple service health check. Returns {"status": "ok"}.

3. Authentication

3.1 Register

- **Method:** POST
- **Path:** /api/v1/auth/register
- **Description:** Create a new user account.

Request body:

- **username** - string, required, 3 to 50 characters
- **email** - string, optional, valid email address
- **password** - string, required, 6 to 128 characters

Successful response:

- **201 Created**
- Returns the created user object with **id**, **username**, **email**, **is_active**, **is_admin**, and **created_at**

Common errors:

- **400 USER_EXISTS** - username or email already registered
- **422** validation error - request body does not satisfy schema rules

3.2 Login

- **Method:** POST
- **Path:** /api/v1/auth/login
- **Description:** Authenticate a user and return a JWT access token.

Request format:

- `application/x-www-form-urlencoded`
- Fields: `username`, `password`

Successful response:

```
{
  "access_token": "<JWT>",
  "token_type": "bearer"
}
```

Common errors:

- **401 INVALID_CREDENTIALS** - incorrect username or password

3.3 Current user

- **Method:** GET
- **Path:** `/api/v1/auth/me`
- **Auth:** required
- **Description:** Return the currently authenticated user.

Common errors:

- **401** - missing or invalid token

4. Authors

Create, update, and delete operations require an authenticated admin user.

4.1 List authors

- **Method:** GET
- **Path:** `/api/v1/authors`

Query parameters:

- **skip** - integer, default 0
- **limit** - integer, default 20, maximum 100
- **search** - optional substring filter on author name

Response:

- **200 OK**
- Array of author objects with `id`, `name`, `country`, and `biography`

4.2 Create author

- **Method:** POST
- **Path:** /api/v1/authors
- **Auth:** admin required

Request body:

- **name** - string, required
- **country** - string, optional
- **biography** - string, optional

Common errors:

- **403** - insufficient permissions
- **422** - schema validation failure

4.3 Get author

- **Method:** GET
- **Path:** /api/v1/authors/{author_id}

Common errors:

- **404** AUTHOR_NOT_FOUND

4.4 Update author

- **Method:** PUT
- **Path:** /api/v1/authors/{author_id}
- **Auth:** admin required

Common errors:

- **403** - insufficient permissions
- **404** AUTHOR_NOT_FOUND

4.5 Delete author

- **Method:** DELETE
- **Path:** /api/v1/authors/{author_id}
- **Auth:** admin required

Successful response:

- **204 No Content**

Behaviour note:

- Deleting an author keeps associated books in the database and sets their `author_id` to `null`
-

5. Books

5.1 List books

- **Method:** GET
- **Path:** /api/v1/books

Query parameters:

- `skip` and `limit` for pagination
- `search` for title substring matching
- `author_id` to filter by author
- `genre` to filter by `main_genre`
- `order_by` with values `title`, `rating`, or `ratings_count`

Response:

- **200 OK**
- Array of book objects including `title`, `isbn`, `publication_date`, `main_genre`, `author_id`, `average_rating`, and `ratings_count`

5.2 Create book

- **Method:** POST
- **Path:** /api/v1/books
- **Auth:** admin required

Common errors:

- **403** - insufficient permissions
- **404 AUTHOR_NOT_FOUND** - referenced author does not exist
- **400 BOOK_ISBN_EXISTS** - duplicate ISBN
- **422** - schema validation failure

5.3 Get book

- **Method:** GET
- **Path:** /api/v1/books/{book_id}

Common errors:

- **404 BOOK_NOT_FOUND**

5.4 Update book

- **Method:** PUT
- **Path:** /api/v1/books/{book_id}
- **Auth:** admin required

Common errors:

- **403** - insufficient permissions
- **404 BOOK_NOT_FOUND** or **AUTHOR_NOT_FOUND**
- **400 BOOK_ISBN_EXISTS**

5.5 Delete book

- **Method:** DELETE
- **Path:** /api/v1/books/{book_id}
- **Auth:** admin required

Successful response:

- **204 No Content**

5.6 Recalculate book rating

- **Method:** POST
- **Path:** /api/v1/books/{book_id}/recalculate-rating
- **Auth:** admin required
- **Description:** Recompute `average_rating` and `ratings_count` from stored reviews.

Common errors:

- **403** - insufficient permissions
 - **404** BOOK_NOT_FOUND
-

6. Reviews

6.1 List reviews

- **Method:** GET
- **Path:** /api/v1/reviews

Query parameters:

- `skip` and `limit`
- `book_id`
- `user_id`

6.2 Create review

- **Method:** POST
- **Path:** /api/v1/reviews
- **Auth:** required

Request body:

- `book_id` - integer, required
- `rating` - integer, required, 1 to 5
- `review_text` - string, optional

Behaviour note:

- Creating a review automatically updates the target book's derived rating fields

Common errors:

- **401** - not authenticated
- **404** BOOK_NOT_FOUND
- **422** - invalid rating or malformed request

6.3 Update review

- **Method:** PUT
- **Path:** /api/v1/reviews/{review_id}
- **Auth:** review owner or admin

Behaviour note:

- Updating a review recalculates the book's rating summary

Common errors:

- **403** REVIEW_FORBIDDEN
- **404** REVIEW_NOT_FOUND

6.4 Delete review

- **Method:** DELETE
- **Path:** /api/v1/reviews/{review_id}
- **Auth:** review owner or admin

Behaviour note:

- Deleting a review recalculates the book's rating summary

Common errors:

- **403** REVIEW_FORBIDDEN
- **404** REVIEW_NOT_FOUND

7. Reading List

All reading-list operations apply to the currently authenticated user.

7.1 List my reading list

- **Method:** GET
- **Path:** /api/v1/reading-list
- **Auth:** required

Query parameters:

- skip
- limit

7.2 Add to reading list

- **Method:** POST
- **Path:** /api/v1/reading-list
- **Auth:** required

Request body:

- book_id - integer, required

Behaviour note:

- If the selected book already exists in the user's reading list, the endpoint returns the existing entry rather than creating a duplicate

Common errors:

- **401** - not authenticated
- **404** BOOK_NOT_FOUND

7.3 Remove from reading list

- **Method:** DELETE
- **Path:** /api/v1/reading-list/{entry_id}
- **Auth:** required

Common errors:

- 403 READING_LIST_FORBIDDEN
 - 404 READING_LIST_ENTRY_NOT_FOUND
-

8. Analytics and Recommendations

8.1 Book rating distribution

- **Method:** GET
- **Path:** /api/v1/analytics/books/{book_id}/rating-distribution
- **Description:** Return the count of 1-star to 5-star ratings for a given book.

Example response:

```
{
  "book_id": 1,
  "distribution": {
    "1": 0,
    "2": 1,
    "3": 2,
    "4": 5,
    "5": 3
  }
}
```

Common errors:

- 404 BOOK_NOT_FOUND

8.2 Top-rated genres

- **Method:** GET
- **Path:** /api/v1/analytics/genres/top-rated

Query parameters:

- **min_ratings** - integer, default 10
- **limit** - integer, default 10, maximum 50

Response:

- Array of genre summary objects with **genre**, **ratings_count**, and **average_rating**

8.3 Trending authors

- **Method:** GET
- **Path:** /api/v1/analytics/authors/trending

Query parameters:

- **days** - integer, default 180
- **limit** - integer, default 10, maximum 50

Response:

- Array of objects including **author_id**, **ratings_count**, **overall_avg**, **recent_avg**, and **trend_score**

8.4 User recommendations

- **Method:** GET
- **Path:** /api/v1/analytics/users/me/recommendations
- **Auth:** required

Description:

- Returns content-based recommendations using the current user's prior high-rated genres, or global top-rated books if the user has not reviewed anything yet

Response:

- Array of recommendation objects with **book_id**, **title**, **average_rating**, **ratings_count**, and **main_genre**

9. HTTP Status Codes Summary

- 200 - successful read, update, or analytical response
- 201 - successful resource creation
- 204 - successful deletion with no response body
- 400 - business-rule failure such as duplicate ISBN or duplicate user
- 401 - missing or invalid credentials
- 403 - authenticated but not permitted
- 404 - requested resource not found
- 422 - request validation failure
- 500 - unexpected internal server error